

Exactness of Credal Variable Elimination on Chain-Graph LCNs

What CVE actually bounds, when the per-target bucket schedule is exact across chains, trees, polytrees, DAGs and chain graphs, and how the tightening schemes (D1–D5) act on it

IBM Research

Abstract

Credal Variable Elimination (CVE, `lcn/inference/marginal/cn/cve.py`) is the hub of the credal-network family: the per-target bucket-elimination engine against which Credal Cluster Tree Elimination (CCTE), ApproxLP, Interval BP and Credal IJGP are all measured. This note pins down three things about CVE against the implementation. **(i) What it bounds.** CVE optimizes over the *strong extension* $\mathcal{D}_{SE}$ of the compiled credal network, not the LCN’s distribution set $\mathcal{D}_{LCN}$. More precisely, CVE does not merely return a valid outer bound over $\mathcal{D}_{SE}$: it returns *exactly* the strong-extension interval $[\underline{q}_{SE}, \bar{q}_{SE}]$, because it recomputes an elimination order per query with the target eliminated *last*, so the extreme-vertex combination that attains a bound survives to the final bucket. Since $\mathcal{D}_{LCN} \subseteq \mathcal{D}_{SE}$, the reported interval is always a valid outer bound on $\mathcal{D}_{LCN}$, and it is the *tightest* such bound any strong-extension propagation can give. **(ii) Which tightening schemes apply.** The five schemes of the companion note (`docs/tighter_approximation.tex`) split for CVE exactly as for CCTE: D1/D2/D3 are *build-stage* knobs on `CredalNetworkVertices.from_lcn` that CVE inherits for *free* (it merely consumes tighter vertices, and being exact over $\mathcal{D}_{SE}$ it reports the tighter strong extension verbatim); D4 (`coupling="cross-family"`) is *implemented inside* CVE as a prune-time function filter *together with* an order augmentation that forces the coupled atoms into a common bucket; D5 is a *separate* engine (CredalJT), not a CVE knob. **(iii) When it is exact.** Because CVE is exact over $\mathcal{D}_{SE}$, its exactness with respect to $\mathcal{D}_{LCN}$ reduces to a single geometric question: *when does $\mathcal{D}_{SE} = \mathcal{D}_{LCN}$?* We show this holds on a clean singly-connected, cross-family-sentence-free LCN — Markov chain, tree, polytree, and chain graph with compound nodes (`alarm`) — so CVE is exact there; and we exhibit the precise boundary — a cross-family *sentence* (`d4.biting`) — where $\mathcal{D}_{SE} \supsetneq \mathcal{D}_{LCN}$ and CVE’s `coupling="off"` interval, though still a valid and exact $\mathcal{D}_{SE}$ bound, is looser than $\mathcal{D}_{LCN}$. On that instance the D4 order augmentation lets CVE reach the exact bound for the coupled *conditional* query while remaining sound, and CredalJT (D5) is the reliable exact engine in general. This last point contrasts sharply with CCTE, whose single shared order makes even the *unconditional* bound fall strictly *inside* $[\underline{q}_{SE}, \bar{q}_{SE}]$ on the same instance.

Contents

1	Background: the CVE algorithm	2
1.1	LCNs, the credal network, and the strong extension	2
1.2	The per-target bucket schedule	2
2	What CVE bounds: exact over the strong extension	3
3	When CVE is exact with respect to $\mathcal{D}_{LCN}$	4
3.1	Topology by topology	4
3.2	A subtle boundary: a marginal on a non-root child	5

4	The boundary: a cross-family sentence (d4.biting)	6
4.1	On the <i>unconditional</i> marginals CVE is still exact here	6
4.2	Where the gap bites: conditional queries and D4	7
5	The tightening schemes (D1–D5) for CVE	7
6	Verdict	8

1 Background: the CVE algorithm

1.1 LCNs, the credal network, and the strong extension

An LCN over binary atoms $\mathbf{V} = \{V_1, \dots, V_n\}$ is a set of probability *sentences*, each Type-1 $\ell \leq P(\varphi) \leq u$ or Type-2 $\ell \leq P(\varphi \mid \psi) \leq u$, over propositional formulas. Together with the conditional independencies of the *Local Markov Condition* (LMC) of the primal graph, the sentences define a set $\mathcal{D}_{\text{LCN}} \subseteq \Delta(2^{\mathbf{V}})$ of joint distributions. The marginal task for a query atom a is $\underline{P}(a) = \min_{p \in \mathcal{D}_{\text{LCN}}} P(a)$ and $\bar{P}(a) = \max_{p \in \mathcal{D}_{\text{LCN}}} P(a)$; exact inference (`ExactInference`, `exact.py`) solves one nonconvex NLP over the full 2^n joint per bound.

The `cn/` pipeline instead compiles the chain-graph LCN into a *credal network*: a directed graph over the families $\text{ch}_v \mid \text{pa}_v$ of the simplified structure graph, whose local credal sets $K_v = K(\text{ch}_v \mid \text{pa}_v)$ are interval-valued and computed per family. `CredalNetworkVertices.from_lcn` enumerates the extreme points of every K_v (`cnv.extreme_points`), and *all* the credal-network engines — CVE, CCTE, Interval BP, ApproxLP and Credal IJGP — consume exactly that object. What they bound is the *strong extension*

$$\mathcal{D}_{\text{SE}} = \left\{ \prod_v P_v : P_v \in K_v \text{ chosen independently per parent-config} \right\}, \quad (1)$$

the set of joints obtained by letting each family, and each parent configuration of each family, pick one extreme point of its local set freely and independently. Because the chain-graph factorization $\prod_v P(\text{ch}_v \mid \text{pa}_v)$ reproduces the LMC exactly (this is what `verify_lmc_factorization.py` certifies), every element of $\mathcal{D}_{\text{SE}}$ satisfies every LMC assertion. Hence

$$\mathcal{D}_{\text{LCN}} \subseteq \mathcal{D}_{\text{SE}}, \quad \implies \quad [\underline{q}_{\text{LCN}}, \bar{q}_{\text{LCN}}] \subseteq [\underline{q}_{\text{SE}}, \bar{q}_{\text{SE}}], \quad (2)$$

so any engine that brackets $\mathcal{D}_{\text{SE}}$ produces a valid *outer* bound on $\mathcal{D}_{\text{LCN}}$. The inclusion in (2) is strict exactly when a *cross-family sentence* (a sentence whose atom scope is contained in no single family scope) couples choices that the free product of (1) decouples. Sections 3 and 4 turn on this distinction.

1.2 The per-target bucket schedule

`CredalVE`’s public `run(evidence, elim_heuristic, epsilon, coupling, n_clusters, cluster_representation, verbosity)` (`cve.py:84--165`) computes the bounds of *every* non-evidence singleton atom by *looping* a single-query bucket elimination over the credal-network nodes. For each target node it calls `run_single_query` (`cve.py:290--482`), which:

1. **Builds initial potentials from the extreme points** (`cve.py:320--381`). Each family becomes a set of CPT-shaped functions: one function per Cartesian combination of a chosen extreme point in every parent configuration. A `Potential` is a finite *set* of non-negative arrays over a shared scope; this set is precisely an explicit vertex enumeration of the family’s contribution to $\mathcal{D}_{\text{SE}}$.

2. **Chooses an elimination order in which the target is last** (`cve.py:391--410`). With `elim_heuristic="topological"` it takes the DAG topological order and removes the query; with `"min-fill"` it runs the greedy interaction-graph heuristic `min_fill_order(scopes, exclude={query})`. Either way the query node’s own bucket *survives* to the end.
3. **Runs one bucket-elimination pass** (`cve.py:421--455`): for each variable in the order, collect the potentials mentioning it, *combine* them (elementwise product, scope union), *marginalize* the variable out, and *prune* dominated functions (g dominates f iff $g \geq f$ everywhere and $g > f$ somewhere; `potentials.py:124--148`).
4. **Reads off the target bounds** (`cve.py:469--481`): every surviving function f over the query scope is normalized to $f / \sum f$, and the per-state min/max across all surviving functions gives $[\underline{P}(\text{query}), \overline{P}(\text{query})]$.

The two decisive features are that (a) each potential is an explicit set of strong-extension vertices, and (b) the query is eliminated *last*. Feature (a) is what makes CVE a *strong-extension* method; feature (b) is what makes it *exact* over $\mathcal{D}_{\text{SE}}$, and is exactly what CCTE’s single shared two-pass order gives up (`docs/ccte_exactness.tex`). An optional $\epsilon > 0$ activates `_run_single_query_approx` (`cve.py:484`), an FPTAS that permits ϵ -domination in pruning (Mauá et al. 2012); with $\epsilon = 0$ the pruning is exact. A second, orthogonal approximation knob `n_clusters > 0` (with `cluster_representative` $\in \{\text{"plub"}, \text{"mean"}\}$) applies the same K-means potential-clustering that CCTE offers (`Potential.cluster_prune`) before each prune, capping the intermediate potential size; like ϵ , it makes the result an *approximation* (no longer exact over $\mathcal{D}_{\text{SE}}$). The default `n_clusters = 0` leaves the exact behavior below unchanged.

2 What CVE bounds: exact over the strong extension

Proposition 1 ($\mathcal{D}_{\text{SE}}$ target). *With `coupling="off"` and exact pruning ($\epsilon = 0$), **CredalVE** computes, for every query atom a , exactly the strong-extension interval $[q_{\text{SE}}(a), \bar{q}_{\text{SE}}(a)] = [\min_{P \in \mathcal{D}_{\text{SE}}} P(a), \max_{P \in \mathcal{D}_{\text{SE}}} P(a)]$. In particular it is always a valid outer bound on $\mathcal{D}_{\text{LCN}}$.*

Proof sketch. Every function CVE forms is a strong-extension point. The initial potentials (`cve.py:320--381`) are, by construction, the extreme points of the families, i.e. the generators of $\mathcal{D}_{\text{SE}}$ in (1). The two potential operations preserve this: `combine` takes an elementwise product of one function from each argument (a product of local choices, still a point of the joint over the union scope), and `marginalize` sums a variable out (the marginal of such a product). Pruning removes only *dominated* functions, which can never attain a componentwise extremum, so it changes no bound. Hence at the final step every surviving function over the query scope is the query-marginal of some $P \in \mathcal{D}_{\text{SE}}$, and the min/max read-off (`cve.py:469--481`) ranges over exactly $\{P(a) : P \in \mathcal{D}_{\text{SE}}\}$ at its extreme vertices.

The query-last order loses no extremum. Because $\mathcal{D}_{\text{SE}}$ is a product of finitely many polytopes, $\max_{P \in \mathcal{D}_{\text{SE}}} P(a)$ is attained at a vertex of that product, i.e. at a specific combination of family extreme points. Eliminating the query *last* means that combination is never marginalized away before the read-off, and it is never pruned, because at the query scope it is (by definition of the extremum) not dominated. Thus the read-off sees it, and CVE returns the exact $\mathcal{D}_{\text{SE}}$ bound. (2) then gives the outer-bound guarantee on $\mathcal{D}_{\text{LCN}}$. $\square$

This is the sense in which CVE is the “gold” member of the family (`docs/tighter_approximation.tex`, comparison table): it is exact over $\mathcal{D}_{\text{SE}}$, whereas ApproxLP is *inner*, Interval BP and IJGP carry

loopy outer slack, and CCTE is exact over $\mathcal{D}_{\text{SE}}$ only when its single shared order happens to coincide with a query-last order — which fails under cross-family coupling (Section 4).

3 When CVE is exact with respect to $\mathcal{D}_{\text{LCN}}$

By Proposition 1, CVE returns $[q_{\text{SE}}, \bar{q}_{\text{SE}}]$ *exactly*. Its exactness with respect to the LCN — $[q_{\text{SE}}, \bar{q}_{\text{SE}}] = [q_{\text{LCN}}, \bar{q}_{\text{LCN}}]$ — therefore reduces entirely to the geometric question of **when** $\mathcal{D}_{\text{SE}} = \mathcal{D}_{\text{LCN}}$. The following proposition characterizes the clean case; the topology enters only through singly-connectedness of the primal graph.

Proposition 2 (Exactness on clean singly-connected LCNs). *Let $\mathcal{L}$ be a chain-graph LCN whose primal graph is singly connected (Markov chain, rooted tree, polytree, or a chain graph whose compound nodes leave the family DAG singly connected, e.g. `alarm`) and which has no cross-family sentence (every sentence’s atom scope fits inside one family; only the LMC assertions may be cross-family). Then with `coupling="off"` and $\epsilon = 0$, `CredalVE` returns the exact LCN marginal bounds for every atom.*

Proof sketch. There are two gaps between the CVE output and the LCN truth, and both close.

(1) $\mathcal{D}_{\text{SE}} = \mathcal{D}_{\text{LCN}}$ (no gap B). A chain-graph product $\prod_v P(\text{ch}_v \mid \text{pa}_v)$ satisfies every LMC assertion by construction, so a free product in (1) can leave $\mathcal{D}_{\text{LCN}}$ *only* by violating a *sentence*. When every sentence is intra-family, each family’s local credal set K_v already encodes every sentence constraining its scope, and the free product of compliant families is again compliant. Hence $\mathcal{D}_{\text{SE}} = \mathcal{D}_{\text{LCN}}$; equivalently, there is no cross-family coupling for the free product of (1) to break. (This is precisely the content certified by `verify_lmc_factorization.py` on chain/tree/polytree instances.)

(2) *CVE is exact over $\mathcal{D}_{\text{SE}}$ (no schedule loss)*. This is Proposition 1: the query-last per-target order guarantees the extreme vertex combination survives to the read-off. Singly-connectedness is not needed for this half — CVE’s per-target order is exact over $\mathcal{D}_{\text{SE}}$ on *any* topology — but it is what makes half (1) hold, so both halves together give exactness w.r.t. $\mathcal{D}_{\text{LCN}}$.

Combining, $[q_{\text{SE}}, \bar{q}_{\text{SE}}] = [q_{\text{LCN}}, \bar{q}_{\text{LCN}}]$ and CVE reports it exactly. $\square$

3.1 Topology by topology

Markov chain. The primal graph is a path $x_0 - x_1 - \dots - x_n$; every family is $P(x_{i+1} \mid x_i)$ with a single parent. Singly connected, no cross-family sentence, so $\mathcal{D}_{\text{SE}} = \mathcal{D}_{\text{LCN}}$ and CVE is exact. On `examples/chain.lcn` ($n = 8$) CVE returns $P(x_0) = [0.300, 0.500]$, $P(x_2) = [0.328, 0.640]$, $P(x_7) = [0.334, 0.667]$, matching `ExactInference` and `CredalJT` (cf. `docs/tighter_approximation.tex`, § chain, and `docs/ccte_exactness.tex`).

Tree. A rooted branching structure in which every node has at most one parent. Same argument: single-parent families, singly connected, no cross-family sentence $\Rightarrow \mathcal{D}_{\text{SE}} = \mathcal{D}_{\text{LCN}} \Rightarrow$ exact. Confirmed on `examples/tree.lcn`.

Polytree. Singly connected but with colliders (a node with several parents). The families are now $P(\text{ch} \mid \text{pa}_1, \dots, \text{pa}_k)$, and moralizing the colliders induces co-parent independencies in the LMC. As long as every *sentence* is intra-family, $\mathcal{D}_{\text{SE}} = \mathcal{D}_{\text{LCN}}$ still holds (the free product respects the moralized LMC), so CVE is exact. Confirmed on `examples/polytree.lcn`. (This is the same boundary ARIEL sits on — `docs/ariel_exactness.tex` — exact on singly-connected, and the LMC is preserved structurally along the unique family path.)

Chain graph with compound nodes. `examples/alarm.lcn` has five atoms and a compound node $C\text{-}D$; the families are $P(A \mid B, E)$, $P(B)$, $P(E)$, $P(C\text{-}D \mid A)$. The two large-scope LMC assertions ($C \perp\!\!\!\perp B, E \mid D, A$) and ($D \perp\!\!\!\perp B, E \mid C, A$) are cross-family, but they are *LMC assertions, not sentences*: the factorization reproduces them for free. There is no cross-family *sentence*, so $\mathcal{D}_{\text{SE}} = \mathcal{D}_{\text{LCN}}$ and CVE is exact, returning e.g. $P(A) = [0.087, 0.100]$, $P(B) = [0.100, 0.123]$, $P(E) = [0.050, 0.074]$ (matching the brute-force oracle, and more reliable than raw `ExactInference`, which can locally over-tighten — `docs/tighter_approximation.tex`, `alarm`).

General / loopy DAG, or any cross-family *sentence*. When the primal graph has more than one path between two atoms (loopy), or a sentence couples atoms across families, $\mathcal{D}_{\text{SE}} \supsetneq \mathcal{D}_{\text{LCN}}$ can be strict: the free product of (1) realizes joints that violate the coupling. CVE still returns the *exact* $\mathcal{D}_{\text{SE}}$ interval (Proposition 1), hence a valid outer bound, but it is no longer tight. This is the boundary of Section 4, and the regime where the coupling schemes D4/D5 earn their keep.

Topology	singly conn.?	cross-family <i>sentence</i> ?	CVE (coupling="off")
Markov chain (chain)	yes	no	exact
Tree (tree)	yes	no	exact
Polytree (polytree)	yes	no	exact
Chain graph, compound (alarm)	yes	no	exact
Cross-family sentence (d4_biting)	yes	yes	exact over $\mathcal{D}_{\text{SE}}$; outer on $\mathcal{D}_{\text{LCN}}$
Loopy DAG	no	—	exact over $\mathcal{D}_{\text{SE}}$; outer on $\mathcal{D}_{\text{LCN}}$

3.2 A subtle boundary: a marginal on a non-root child

The “no cross-family sentence” condition is about a sentence’s *atom scope* not fitting in one family. But a sentence can behave as an *effective* cross-family constraint even when its atom scope is a single atom that trivially fits in a family. The case is a **Type-1 marginal $P(x)$ on a non-root child x** (an atom with at least one parent). In the factorization x ’s family is the conditional $P(x \mid \text{pa}_x)$, so its marginal is $P(x) = \sum_{\text{pa}_x} P(x \mid \text{pa}_x) P(\text{pa}_x)$ — a functional that couples x ’s family with its parents’ marginals *across* families. No single per-family local credal set can enforce the bound on $P(x)$: the family $P(x \mid \text{pa}_x)$ constrains only the conditional rows, and treats $P(\text{pa}_x)$ as given. So the strong extension $\mathcal{D}_{\text{SE}}$ lets the composed marginal $P(x)$ drift past the sentence’s bound (Gap B), and CVE (exact over $\mathcal{D}_{\text{SE}}$) reports the inflated interval, even though the primal graph is a clean singly-connected tree. Crucially, the atom-scope cross-family test sees only the single atom $\{x\}$ (which fits every family containing x) and reports the sentence as family-local, so D4 does *not* flag or repair it.

Witness (`benchmarks/tree_small/tree_n5.1.lcn`, a 5-atom tree $x_3 \rightarrow x_2 \rightarrow x_0$, $x_3 \rightarrow x_4 \rightarrow x_1$, with the extra marginal $s_5 : P(x_2) \leq 0.4395$ on the non-root child x_2): the exact bound is $P(x_2) = [0.010, 0.4395]$ (`CredalJT` and `ARIEL` agree, hitting s_5 exactly), but composing the local extreme points as $\mathcal{D}_{\text{SE}}$ does gives $P(x_2) \in [0, 0.8012]$ — exactly CVE’s and Interval BP’s output, grossly violating s_5 . The looseness then propagates downstream: $P(x_0)$ inflates from the exact $[0.006, 0.621]$ to $[0.002, 0.865]$ (its family $P(x_0 \mid x_2)$ additionally leaves the $x_2=1$ row vacuous, having only $s_4 : P(x_0 \mid \neg x_2)$). Neither D1 (**linear-tight**) nor the atom-scope D4 closes the gap, because the coupling is genuinely across families; only D2 (merging the families) or the exact engine (`CredalJT`, D5) recovers the bound.

The family-realizable benchmark classes. The benchmark generator (`lcn/benchmarks/generator.py`) exposes dedicated *family-realizable* tree and polytree classes, `graph_type="tree-fr" / "polytree-fr"`

(CLI: `--family-realizable` on `gen_trees.py` / `gen_polytrees.py`, or `--types tree-fr polytree-fr` on `gen_benchmarks.py`). These place the extra marginal sentences (`num_extras`) *only on root atoms* (atoms with no parents, whose family *is* the marginal $P(x)$ and hence enforces the bound in-family), so generated `-fr` instances lie in the clean $\mathcal{D}_{\text{SE}} = \mathcal{D}_{\text{LCN}}$ regime and CVE / Interval BP / ARIEL are exact on them, matching `CredalJT`. The default `tree` / `polytree` classes draw extras from all atoms and so may contain a non-root marginal (as may hand-authored LCNs like `tree_n5_1.lcn`); there, use `CredalJT` for the exact bound. The companion note `docs/strong_extension_exactness.tex` formalizes this: it defines the *family-realizable* sentence (a constraint on one family’s conditional alone) and proves that CVE is exact w.r.t. $\mathcal{D}_{\text{LCN}}$ *iff* the primal graph is singly connected and every sentence is family-realizable — the sharp condition of which “no cross-family sentence” is the atom-scope-visible half. A non-root marginal is one non-realizable form; a second is a *collider* conditioned on a multi-configuration formula, $P(x_2 \mid x_1 \oplus x_3)$ (binding through the co-parents’ joint), which is likewise loose for CVE regardless of solver or scheme D1 — see that note for the full catalogue and the exact route (D2 merge or `CredalJT`).

4 The boundary: a cross-family sentence (d4_biting)

The minimal witness that separates $\mathcal{D}_{\text{SE}}$ from $\mathcal{D}_{\text{LCN}}$ is `examples/d4.biting.lcn`: three atoms, A the root with independent children B, C , so the families are $P(A)$, $P(B \mid A)$, $P(C \mid A)$, plus one cross-family sentence.

```
A1: 0.4 <= P(A)          <= 0.6
B1: 0.1 <= P(B | A)      <= 0.3
C1: 0.2 <= P(C | A)      <= 0.5
BC: 0.0 <= P(B and C)    <= 0.05      <- cross-family (scope {B,C} in no family)
```

The LMC assertion $(B \perp\!\!\!\perp C \mid A)$ is satisfied by *any* product joint, so it is not what makes this instance loose; the *sentence* BC is. The free product of (1) can pick a high- $P(B \mid A)$ vertex and a high- $P(C \mid A)$ vertex at once, reaching $P(B \wedge C)$ far above 0.05 — a joint in $\mathcal{D}_{\text{SE}}$ but not in $\mathcal{D}_{\text{LCN}}$.

4.1 On the *unconditional* marginals CVE is still exact here

The unconditional bounds happen to be unaffected by the coupling gap on this instance: the exact $\mathcal{D}_{\text{LCN}}$ intervals are

$$P(A) = [0.40, 0.60], \quad P(B) = [0.04, 0.72], \quad P(C) = [0.08, 0.80],$$

confirmed by four independent oracles (parametric SLSQP enforcing $B \perp\!\!\!\perp C \mid A$; a strong-extension vertex oracle; `CredalJT`/D5 with SCIP; and single-query CVE — `docs/ccte_exactness.tex`). CVE with `coupling="off"` returns exactly $P(B) = [0.04, 0.72]$ because, for the *unconditional* $P(B)$, the extremal vertex combination does not require violating BC — so $\mathcal{D}_{\text{SE}}$ and $\mathcal{D}_{\text{LCN}}$ agree on this particular marginal, and CVE’s exact $\mathcal{D}_{\text{SE}}$ read-off (Proposition 1) is the exact LCN bound.

	$P(A)$	$P(B)$	$P(C)$
exact = <code>CredalVE</code> = oracle = D5	[0.40, 0.60]	[0.04, 0.72]	[0.08, 0.80]
CCTE <code>coupling="off"</code>	[0.40, 0.60]	[0.05, 0.65]	[0.08, 0.80]
CCTE <code>coupling="cross-family"</code> (D4)	[0.40, 0.60]	[0.05, 0.55]	[0.08, 0.80]

The contrast with CCTE is the whole point of the per-target schedule. CCTE, using the *same* vertices, reports $P(B) = [0.05, 0.65]$ — strictly *inside* the true $\mathcal{D}_{\text{SE}}$ interval, i.e. not even a valid outer

bound — because its single shared order eliminates B mid-schedule and prunes (by dominance) the very function that carried the extremal $P(B)$; dominance in a marginalized scope is not dominance for the eventual read-off. CVE avoids this precisely by recomputing a query-last order for B , so the extremal function survives (Proposition 1). This is the “order-sensitive outer approximation” the code comment warns about, and it is CVE’s per-target recomputation that immunizes it.

4.2 Where the gap bites: conditional queries and D4

The coupling gap $\mathcal{D}_{\text{SE}} \supsetneq \mathcal{D}_{\text{LCN}}$ does bite on *conditional* queries. The `.lcn` header records that $P(B \mid C = 1)$ collapses from $[0.0375, 0.9625]$ under `coupling="off"` to $[0.0375, 0.0375]$ under `coupling="cross-family"`: the free product allows B and C both near their high vertices given A , which the sentence BC forbids. Scheme D4 (Section 5) removes exactly those vertex *combinations*. In CVE this is two coordinated changes (`cve.py:391--455`):

1. **Order augmentation.** The min-fill order is built from the potential scopes *plus* the constraint node sets, `scopes + constraints.constraint_node_sets(...)` (`cve.py:408--410`), forcing the coupled atoms $\{A, B, C\}$ to co-occur in a common bucket. Without this the constraint scope is never assembled and the filter is inert.
2. **Prune-time filtering.** In each bucket, *after* combining and *before* marginalizing, `combined.filter_infeasible` (`cve.py:441--442`) drops any function whose assembled joint violates a cross-family constraint — checkable only once the bucket scope covers the constraint’s atoms, which the augmentation guarantees.

D4 is sound for *unconditional* marginals (the extremum sits at a surviving vertex). For conditional $P(q \mid e)$ the ratio extremum need not sit at a vertex, so D4 is not guaranteed exact; the reliable exact engine for cross-family coupling is **CredalJT** (D5). Note also that on the *unconditional* $P(B)$ above, D4 is *correctly inert*: CVE augments its order and the filter fires but finds nothing to drop that would change $[0.04, 0.72]$ — consistent with $\mathcal{D}_{\text{SE}}$ and $\mathcal{D}_{\text{LCN}}$ already agreeing on that marginal.

5 The tightening schemes (D1–D5) for CVE

The companion note `docs/tighter_approximation.tex` identifies two sources of looseness in the strong-extension bound and five schemes that attack them:

- **Gap A (local-set widening).** Each K_v is computed per family in isolation, so it admits distributions no global LCN distribution realizes (it ignores sentences on *other* families and cross-family LMC equalities). This widens $\mathcal{D}_{\text{SE}}$ before any propagation.
- **Gap B (strong-extension widening).** Even with perfect K_v , the free product of (1) lets each family/parent-config pick its vertex independently, violating cross-family sentences. This is the $\mathcal{D}_{\text{SE}} \supsetneq \mathcal{D}_{\text{LCN}}$ gap of Section 4.

Because CVE is exact over $\mathcal{D}_{\text{SE}}$ (Proposition 1), it reports whatever $\mathcal{D}_{\text{SE}}$ it is given *verbatim* — so shrinking $\mathcal{D}_{\text{SE}}$ toward $\mathcal{D}_{\text{LCN}}$ pulls CVE’s output inward with no algorithmic slack of its own to absorb the change. The schemes therefore split into two classes for CVE exactly as they do for CCTE.

Class 1 — input-level (D1, D2, D3): free build-stage knobs. These change only what `CredalNetworkVertices.from_lcn` produces — tighter local intervals, hence tighter enumerated vertices — and CVE consumes them without any code change.

- **D1** (`method="linear-tight"`): each family’s local LP is enriched with every in-scope sentence *and* the scope-restricted LMC bilinear equalities, closing Gap A partially. Vacuous $[0, 1]$ CPT rows collapse to genuine sub-intervals.
- **D2** (`merge_budget=k`): merge adjacent families whose combined scope $\leq k$ before factorization; at $k = 1$ no merge, at $k = n$ exact. Larger merged scopes bring cross-family constraints in-scope, closing Gap A and, via the merge, part of Gap B.
- **D3** (`solver="scip"`): solve each family’s D1 model with SCIP’s certified global branch-and-bound instead of local ipopt, closing Gap A *exactly* (and flagging any unsound ipopt local optimum). Cost 2^n per solve.

Being exact over $\mathcal{D}_{SE}$, CVE is the cleanest beneficiary of Class 1: the tighter strong extension is reported exactly, so D1/D2/D3 pull CVE’s interval straight toward $\mathcal{D}_{LCN}$.

Class 2 — propagation-level coupling (D4, D5): per-algorithm.

- **D4** (`coupling="cross-family"` on `CVE.run`): implemented inside CVE as the order augmentation + prune-time filter of Section 4 (`cve.py:391--455`, `coupling.py:101--250`). It attacks Gap B by dropping infeasible vertex *combinations*. Sound for unconditional marginals; not guaranteed for conditional ratios.
- **D5** (`CredalJT`, `cn/junction_nlp.py`): a *separate* engine, not a CVE knob. It builds one junction tree, introduces cluster-marginal variables, imposes separator-consistency and hosts each LCN constraint on a cluster, and solves one NLP per atom. It closes Gap A + Gap B *exactly* whenever every sentence is hosted and every LMC is hosted-or-separator-entailed (Theorem 1 of `docs/tighter_approximation.tex`), at cost exponential in *treewidth* rather than n . It is sound for both unconditional and conditional queries. For cross-family coupling this is the reliable exact engine.

Scheme	Where it lives for CVE	Gap closed	Exact?	Cost
D1 <code>method="linear-tight"</code>	build (<code>from_lcn</code>)	A (partial)	no	≈ 0
D2 <code>merge_budget=k</code>	build (<code>from_lcn</code>)	A (+ B)	at $k = n$	2^k
D3 <code>solver="scip"</code>	build (<code>from_lcn</code>)	A (exact)	Gap A only	$2^n/\text{solve}$
D4 <code>coupling="cross-family"</code>	inside <code>CVE.run</code>	B	uncond. only	med
D5 <code>CredalJT</code>	separate engine	A + B	yes (Thm. 1)	treewidth

6 Verdict

Proposition 3 (Summary). *Let $\mathcal{L}$ be a chain-graph LCN and run `CredalVE` with $\epsilon = 0$.*

1. (Always) With `coupling="off"`, CVE returns exactly the strong-extension interval $[q_{SE}, \bar{q}_{SE}]$, a valid outer bound on $\mathcal{D}_{LCN}$ (Proposition 1).
2. (Exact) If $\mathcal{L}$ is singly connected and has no cross-family sentence — Markov chain, tree, polytree, or compound-node chain graph such as `alarm` — then $\mathcal{D}_{SE} = \mathcal{D}_{LCN}$ and CVE is exact for every atom (Proposition 2).

3. (Boundary) A cross-family sentence makes $\mathcal{D}_{\text{SE}} \supsetneq \mathcal{D}_{\text{LCN}}$ possible. Unconditional marginals may still be exact (they are on **d4_biting**); conditional queries and the general case need D4 (unconditional-sound) or, for a guaranteed exact bound, the D5 engine **CredalJT**.
4. (Tightening) D1/D2/D3 tighten CVE for free at build time and are reported verbatim (CVE being exact over $\mathcal{D}_{\text{SE}}$); D4 is CVE's built-in coupling filter with order augmentation; D5 is the separate exact engine.

Compared to its siblings: CVE is exact over $\mathcal{D}_{\text{SE}}$ (like CCTE with $\epsilon = 0$, unlike ApproxLP's inner bound and IBP/IJGP's loopy outer slack), and its per-target query-last schedule makes it *strictly more reliable than CCTE* the moment a cross-family sentence appears — where CCTE's single shared order can fall inside the strong-extension interval, CVE always returns it exactly. The three companion notes cover the neighbors: **docs/ccte_exactness.tex** (the two-pass schedule and its **d4_biting** inner error), **docs/ariel_exactness.tex** (structural LMC preservation on singly-connected factor graphs), and **docs/tighter_approximation.tex** (the D1–D5 taxonomy and the D5 exactness theorem).